

Design Document: Pentachess

Version 1.0

Team 6

Brandon Bosman

Mian Rafay

Karl Sader

Saad Tariq

Emaan Khan

Table 1: Revision History

Date	Developer(s)	Change
January 26, 2025	Entire Team	Initial Draft
April 4, 2025	Brandon Bosman, Saad Tariq, Main Rafay	Final Draft

1. Purpose Statement

An online multiplayer chess game featuring the Pentachess variant, with a wild new board and ruleset redesign that presents a new challenge to old and new players alike.

2. Diagram of Components

Figure 1: High-Level Architecture Overview

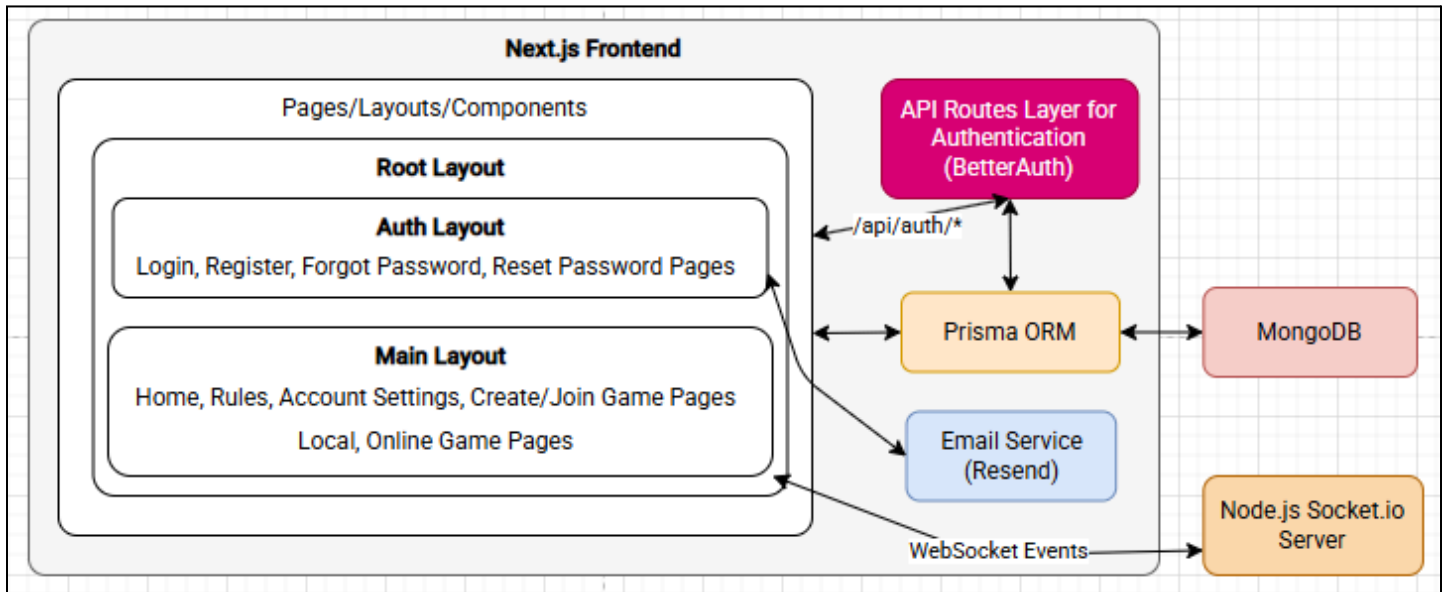
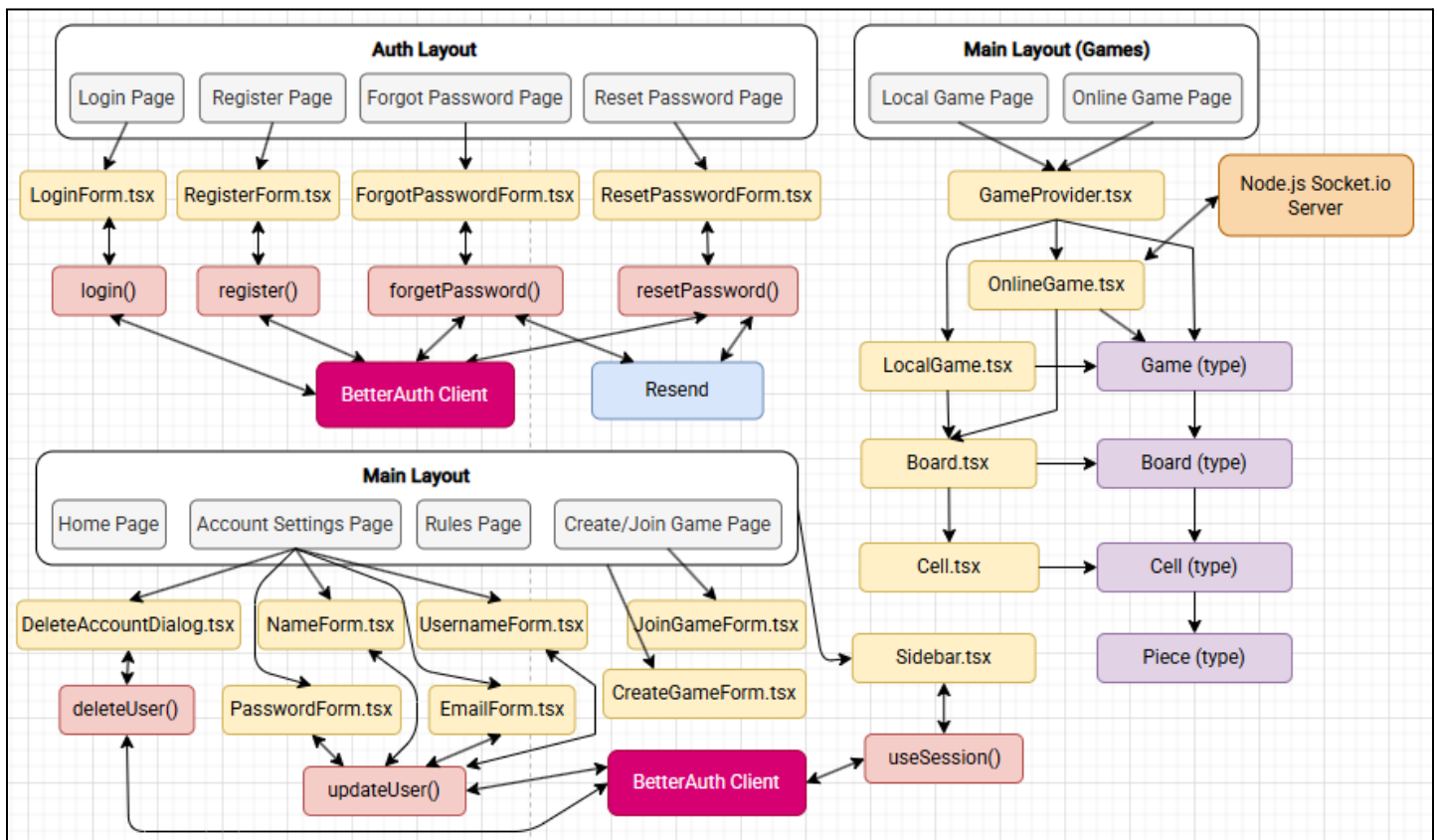


Figure 2: Detailed Overview within Layouts



3. Relationship between Components and Requirements

Highlight where a selected piece can go	
Related Components	<code>Board.tsx</code> , <code>Cell.tsx</code>
To implement this requirement, we created an algorithm to determine all possible moves for every piece within any given cell with the <code>getPossibleMoves()</code> function part of the <code>Cell</code> component. When a user clicks on a piece, the <code>Board</code> component retrieves all possible moves from the <code>Cell</code> component through <code>getPossibleMoves()</code>. The <code>Board</code> component then updates its internal state and changes the colors of all cells that the piece can move to and capture.	
Ability to move pieces from your side	
Related Components	<code>Board.tsx</code> , <code>Cell.tsx</code>
To implement this requirement, we have currently allowed the user to click the piece they want to move and click one of the available cells that the piece can move to, to simulate movement. Internally, the <code>Board</code> component retrieves all possible moves for a selected piece from <code>getPossibleMoves()</code> from the <code>Cell</code> component. All the data and updates to the <code>Board</code> component happen via internal React.js state hooks.	
Store and update the board state when a piece is moved	
Related Components	<code>Board.tsx</code> , <code>GameProvider.tsx</code>
We have a game state, which is the <code>GameProvider</code> component. This component keeps track of the turns, status, timer, captured pieces, previous moves, etc. When a piece is moved, the <code>Board</code> component is updated and is reflected through the UI, and it also updates the <code>GameProvider</code> component through internal state hooks. This <code>GameProvider</code> component keeps track of the board state, previous moves, and captured pieces.	
Expiry of game session upon logout/forfeit/inactivity	
Related Components	<code>GameProvider.tsx</code>
With an internal timer, the <code>GameProvider</code> component keeps track of the time for the user's turn. If no move has occurred within the allotted time, or the opponent/user has forfeited, then a dispatch is sent to the <code>GameProvider</code> component to end the game and the session.	
User Authentication and Account Management	
Related Components	Authentication and Account Setting Components and Functions

The authentication and account settings pages (login, register, forgot password, etc.) and related components and functions, along with the account setting paired with authentication functionality using the **BetterAuth** library with **Prisma ORM** to store user information in **MongoDB**, using session-based authentication, help manage this requirement.

BetterAuth handles preserving authentication states and exposes useful methods via an auth client to manage sessions in both server and client contexts:

- **Server-based contexts:** Methods like `api.getSession()` verify session existence, ensuring users are redirected if not authenticated. They are also used to get the details of the currently logged-in user to update user information, delete accounts, etc.
- **Client-based contexts:** Hooks such as `useSession()` allow forms and components to update session data directly, while exposed utility methods like `login()` (used in the login form submit handler) and `logout()` (triggered via the account button in the sidebar) streamline user interactions.

Internally, **BetterAuth** relies on the catch-all API route (`/api/auth/*`) to handle login, logout session retrieval, etc., as shown in the overview diagram.

4. Component Behaviour

4.1 Existing Pages

Home Page

Normal Behaviour: This home page is the primary entry point to the website, the first page users are greeted with when they navigate to the website. It allows users to easily access the local and online game routes.

Implement: This page includes a title paired with a brief description of the Pentachess variant, an image of the board, and quick action links for starting a local or online game.

Potential Undesired: The content may overflow, or the layout could shift during resizing or on extremely small viewport sizes (less than 400px width). Additionally, components might not render as expected or show incompatibilities with certain browsers.

Rules Page

Normal Behaviour: The rules page is designed to help new players understand the official rules of the Pentachess variant. It provides a comprehensive guide to game mechanics, ensuring players have a solid foundation before starting.

Implement: This page features displays explanations of the basic game mechanics and rules such as check, checkmate, draw, player resignation and timer functionality. It also features an accordion list section featuring images and descriptions for all piece movement scenarios in the moves sections.

Potential Undesired: Same as the [Home Page](#).

Login Page
Components: <code>LoginForm.tsx</code>
Normal Behaviour: The login page contains a form with email and password input fields, which will need to match up with an existing account stored in the database. Both input fields have requirements that the user must meet to attempt a login. Links are also provided to create a new account (Register Page) or to reset the password (Forgot Password Page).
Implement: This page displays a card at the center which contains a title, quick description and the <code>LoginForm.tsx</code> component.
Potential Undesired: Two users logging in with the same account could result in undesired outcomes. We'll need to ensure only 1 user is logged on at a time or enforce account-sharing rules.

Register Page
Components: <code>RegisterForm.tsx</code>
Normal Behaviour: The register page contains a form with username, email, and password fields. Each field has a set of requirements that the user must meet to create an account. Of course, creating an account with an already existing email or username is not possible. A link is also provided to log in to an existing account (Login Page).
Implement: This page displays a card at the center, which contains a title, quick description and the <code>RegisterForm.tsx</code> component.
Potential Undesired: A malicious user could create a multitude of spam accounts that fill up our database with unnecessary user information. However, the BetterAuth library has built-in rate limiting for authentication services, which will help prevent this.

Forgot Password Page
Components: <code>ForgotPasswordForm.tsx</code>
Normal Behaviour: The forgot password page contains a form with an email field. Upon submission, a code will be sent to their email where they can reset their password. A link is also provided to log in to an existing account.
Implement: This page displays a card at the center, which contains a title, quick description and the <code>ForgotPasswordForm.tsx</code> component.
Potential Undesired: If a user creates an account with an email that doesn't exist, they won't be able to reset their password, as resetting it usually requires identification or email access.

Reset Password Page
Components: <code>ResetPasswordForm.tsx</code>
Normal Behaviour: After receiving a reset email with a link, users are redirected to this page, where the URL contains a token used to validate the request. If the token is expired, already used or invalid, users will see a simple card which displays a message.
Implement: This page displays a card at the center which contains a title, a quick description and the <code>ResetPasswordForm.tsx</code> component.
Potential Undesired: Though it will be virtually impossible, a malicious user could, in theory, try to brute force a valid token to have unauthorized access to resetting another user's password.

Account Settings Page
Components: <code>NameForm.tsx</code> , <code>UsernameForm.tsx</code> , <code>EmailForm.tsx</code> , <code>PasswordForm.tsx</code> , <code>DeleteAccountDialog.tsx</code>
Normal Behaviour: The account settings page contains many forms to update important user information such as their username, email address, and password. Users will also be able to delete their accounts if they'd like. It is only available to authenticated users; if a user not logged in tries to access this page, they will be redirected to the login page.
Implement: Has sections which contain the title, description, related form or delete dialog.
Potential Undesired: Same as the Home Page .

Create/Join Game Page
Components: <code>CreateGameForm.tsx</code> , <code>JoinGameForm.tsx</code>
Normal Behaviour: This page consists of two forms to create and join an online game, where users will be redirected to the game page upon successful form submission. It is only available to authenticated users; unauthenticated users will be redirected to the login page.
Implement: This page contains a card in the center which displays the <code>CreateGameForm.tsx</code> , <code>JoinGameForm.tsx</code> , stacked vertically.
Potential Undesired: Same as the Home Page .

Game Page
Components: <code>GameProvider.tsx</code> , <code>Board.tsx</code> , <code>Cell.tsx</code> , <code>Piece</code>
Normal Behaviour: The game page allows the user to play the entire game of Pentachess and perform all in-game actions with insight into key details regarding the game. It allows users to select

cells/pieces and view all possible moves, with the ability to move those pieces and see any capturing or movement motion in real time. The user must wait idly by until their opponent has made a move, for which that move will be reflected on the board. Moreover, all captured pieces and previous moves will be displayed around the board, with the timer for each turn on the left side of the screen.

Local (/game/local): users are shown a card which allows them to pick an optional duration; when they submit, it gets replaced with the game board

Online Random (/game/online): users are shown a card while they wait for an opponent to join the random matchmaking queue.

Online Custom (/game/online/:code): if the game requires a password, they have to enter that, then are shown the game, that is if the game has not already started

Implement: This page consists of these essential components: **GameProvider.tsx**, **Board.tsx** and **Cell.tsx**. Within the game page (local or online game), the **GameProvider.tsx** component keeps track of the game state, including turns, status, timer, captured pieces, previous moves, etc. The **Board.tsx** component organizes and renders the board as sides composed of individual cells. It also keeps track of all available moves for any given piece concerning their cell positioning. Finally, the **Piece** type within a cell holds all metadata regarding the piece type, potential promotions, and visualizations. Together, these components create a dynamic environment that handles all Pentachess gameplay.

Potential Undesired: The content may overflow, or the layout could shift during resizing or on extremely small viewport sizes (less than 400px width). Additionally, components might not render as expected or show incompatibilities with certain browsers. Also, due to the constraints of CSS and the shape of each cell, there may be spacing and positioning discrepancies within the board. Moreover, most in-game logic is handled within the Next.js frontend, however, the timing of the game will be handled within the Node.js backend as well. This may cause inaccurate/inconsistent timing between the user, opponent, and backend. Finally, browser differences could cause the libraries we use to fail on some browsers.

4.2 Existing Components

Authentication: **LoginForm.tsx**, **RegisterForm.tsx**, **ForgotPasswordForm.tsx**, **ResetPasswordForm.tsx**

Normal Behaviour: The user can either "login", "register" or click on the "forgot password" link in this state. Upon clicking the login button, the user is expected to enter a valid email/username and password, which is verified with the backend database for verification. If the user selects to register, they will be prompted to enter a valid email address, a unique username, and a password that meets the requirements displayed on the page. If the user clicks the "Forgot Password" link, they will be prompted to enter a valid email address. A password reset link will then be sent to that email.

Implement: Each authentication component (**LoginForm**, **RegisterForm**, **ForgotPasswordForm**, **ResetPasswordForm**) contains input fields that, when inputted correctly, can be submitted (via a button). When submitted, a **handleSubmit()** function is called that performs each action with the given inputs. The components also contain links to different forms for easy navigation.

Potential Undesired: A malicious user logging into an account that is not theirs. Two-factor authentication could be a solution to ensuring only the account's owner can log in to their account.

Account Settings: NameForm.tsx, UsernameForm.tsx, EmailForm.tsx, PasswordForm.tsx, DeleteAccountDialog.tsx

Normal Behaviour: Each of these form components updates one particular field of a user profile on a successful submission, along with a delete dialog component to confirm account deletion.

Implement: Each form component contains input fields that, when inputted correctly, can be submitted (via a button). The name, username, and email forms each have a single text field, and the password form has two fields: one for the current password and one for the new password, ensuring that the current password is valid for the account and the new password is not the same as the current one. When they are submitted, a `handleSubmit()` function is called that performs each action with the given inputs, using the `updateUser()` authentication function. The dialog component features a button which, when clicked, opens a confirm dialog for deleting an account, cancelling closes the dialog, and confirming uses the `deleteUser()` authentication function.

Potential Undesired: Need to ensure error messages are ambiguous for emails that are already in use to prevent users from trying different emails to see other existing accounts. Deleting an account while a game is in progress might have some unforeseen consequences.

Sidebar.tsx

Normal Behaviour: This component is available for all pages in the main layout and is placed conveniently on the left for easy access.

Implement: Ensure the sidebar contains the logo, theme toggle, and links to pages for quick access like playing local/online, rules page, etc. It also shows a login button for logged-out users and a drop-down with the user image and username for logged-in users, which, when clicked, allows the user to log out or navigate to the account settings page. It will also have a contract/expand button that will allow users to hide or show it as desired.

Potential Undesired: Same as the [Home Page](#).

CreateGameForm.tsx

Normal Behaviour: Allow users who are logged in to create a new game, giving them options to choose the starting side, duration and game password, which are optional.

Implement: Form with optional fields for choosing the starting side, game duration, and password. For online games, upon successful submission, the `createGame` action is used to make a new game instance in the database, and the user is redirected to the newly created game page. For local games, only the duration field is shown, and upon submission, the form updates the game state to playing, thereby replacing the form in place with the board.

JoinGameForm.tsx
Normal Behaviour: Users will be presented with the option to enter an invite code to join a custom game. Upon entering a valid code, they will be redirected to that respective game.
Implement: Form with a field to enter a code featuring a join button, which upon clicking will show errors if the field is missing or invalid, otherwise, it will redirect the user to the active game with that code. This functionality will be handled using the <code>handleSubmit()</code> function within the component.
Potential Undesired: Users can try and brute force different code combinations to try and seek out active games to which they are not invited.

GameProvider.tsx
Components: Board.tsx
Normal Behaviour: The GameProvider component keeps track of the game state, including turns, status, timer, captured pieces, previous moves, and the board state. It communicates with the Board component to relay any updates to and from the Node.js server.
Implement: The GameProvider component initializes and pre-sets the state of the Board component and the entire game with the use of dispatch , context creation , and reducers from React.
Potential Undesired: Data inconsistencies may occur within the game state, specifically regarding the timing of turns (ensuring time is accurate for both players, either local or online), the status of the game regarding checkmates, draws, etc, and the state of the board (regarding positions and promotions of pieces).

Board.tsx
Components: GameProvider.tsx, Cell.tsx, Piece
Normal Behaviour: The Board component organizes and renders the board as sides composed of individual cells. It also keeps track of all available moves for any given piece concerning their cell positioning, displaying previous moves, captured pieces, and a timer. It dispatches any updates to the GameProvider component and receives updates when an opponent makes a move.
Implement: The Board component ensures the correct rendering of all rings, sides, and cells. It contains pre-set internal state configurations from useGame for context, and it ensures that state updates are propagated to the correct cells.
Potential Undesired: Each player has a timer for when they must make a move within their respective turns, and the time value is shared amongst the user, Node.js backend, and opponent. There can be potential inaccuracies with the time value in each aspect of the application stack, which can lead to a decrease in the quality of gameplay, draws, forfeits, or unfair wins.

Cell.tsx
Components: Board.tsx, Piece
Normal Behaviour: The Piece type defines the type of the piece (Bishop, Rook, Pawn, etc), the color of the piece (current user or opponent), the value, and the image of the associated piece (for visual purposes). Moreover, the Cell component also has key functions like getPossibleMoves() , which can generate all the possible moves for a specified piece with the information stored within the Board component using an algorithm.
Implement: The cell component has a parent component (Board) and a child component (Piece). It interacts with the board component to determine the polygon rotation and ensures that the correct piece is rendered within the cell. Using useGame context, with updates using state hooks, ensures that cell metadata is current.
Potential Undesired: There are many states that the Board component can be, and so there are many edge cases that must be accounted for when designing this algorithm to ensure that getPossibleMoves() is accurate. There is a possibility that the algorithm doesn't account for some of these edge cases, which can either break the game and cause a draw or decrease the quality of gameplay for the user.

4.3 Backend (Game, Board, Cell, Piece)

Game Type, States, and Functions
Normal Behaviour: This class represents the game as a whole. It is designed to store the following information: • player<Player>: The information for player 1, which contains the following: ◦ id<string> ◦ userId<string> ◦ username<string> ◦ color<PieceColor> • opponent<Player>: information for player 2, contains same info as above. • winner<PieceColor "draw" null> The winner of the match, if one exists. • turn<PieceColor>: The color of the player whose turn it is. • timer<Map<string, float> null>: The time remaining for each player if a timer has been selected. • previousMoves<Move[]>: A list of previous moves of type Move which contain the following: ◦ notation<string> ◦ timestamp<Date> ◦ piece<string> ◦ from<Cell> ◦ to<Cell> ◦ pieceCaptured<string null> ◦ piecePromoted<string null> ◦ check<boolean> ◦ checkmate<boolean> • capturedPieces<Map<string, Piece[]>>: The captured pieces of each player.

- **check<PieceColor | null>**: Indicates whether a player is in check; if so, it holds the color of the king currently in check
- **status<GameStatus>**: The current state the game is in: "waiting", "playing", "checkmate", "draw-stalemate", "draw-agreement", "draw-threelfold", "draw-fifty-move", "draw-insufficient", "resignation", "timer-expired", or "opponent-left".
- **disabled<boolean>**: Whether the board is disabled for the player in an online match.
- **boardState<BoardState>**: Holds all the properties of the board, such as:
 - All cells which make up the board
 - Currently selected cell and its possible moves depending on the piece type
- **promotionCoordinates<{from<Cell>; to<Cell>; piece<Piece>} | null>**: The information about a pawn that can be promoted.

Implement: The game class takes 3 main arguments, which include player and opponent colors to distinguish between players 1 and 2 as well as the board, which is created by a list of cells. As the games begin, a few things are initialized. Turn is initialized to "w" and rotates between "w" and "b" on every turn to keep track of whose turn it is. The timer is initialized for both colors, starting at a specific time and counting down accordingly per turn. previousMoves and capturedPieces are initially empty, but as pieces are moved, the history is added to previousMoves, and a captured piece is added to capturedPieces. Using the 'check' function at the end of every turn, the game state is calculated to determine if a checkmate or draw has occurred, otherwise, it checks if the timer has expired or a player has resigned. If not, the game state continues to stay as "playing".

Potential Undesired: Incorrect color passed to the resign function might make the opponent resign.

Board Type, States, and Functions

Normal Behaviour: This class represents the chess board. It is designed to store a 2D array of length 3 representing each decagon and various lengths for each array based on the number of cells in the specific decagon.

Implement: The board class takes no arguments and initializes an empty board and iteratively adds cells, calling the **createCell()**. The respective pieces are added to the cells per their starting positions. Lastly, the **setCellEdges()** and **setCellVertices()** are called, which set the edges and vertices of each cell in the board.

Cell Type, States, and Functions

Normal Behaviour: This class represents a single cell (pentagon) on the chessboard. It is designed to store the following information for each cell:

- **id<string>**: A unique identifier for the cell.
- **color<string>**: Indicates whether the cell is light or dark.
- **x<int>**: The x coordinate of the cell's position on the board.
- **y<int>**: The y coordinate of the cell's position on the board.
- **side<int>**: what was this for again lol
- **angle<int>**: The angle that defines the orientation of the cell.
- **piece<Piece | null>**: The chess piece placed on the cell (or null if the cell is empty).

- **edges**<[int,int][]>: The cells which touch edges with the current cell
- **vertices**<[int,int][]>: The cells which touch vertices with the current cell

Implement: The constructor takes the angle of the cell and the coordinates, x and y, where x is the ring and y is the specific cell in that ring. The y coordinate determines the color of the cell, and the id is calculated from the **cellId()**, which takes the x and y coordinates as inputs and returns the corresponding id. The edges and vertices of each cell and the pieces are all initially empty, which are later updated in the board initialization function.

Piece Type, States, and Functions

Normal Behaviour: This class represents a chess piece. It is designed to store the following information for each piece:

- **type**<**PieceType**>: Identifies which piece it is (e.g. "bishop", "rook", etc.).
- **abbr**<**PieceAbbr**>: Abbreviation of the piece's name, used for notation.
- **color**<**PieceColor**>: Indicates whether the piece belongs to the `white` or `black` side.
- **value**<int>: The value of the given piece (e.g. 9 for queen).
- **image**<**StaticImageData**>: The image URL of the respective piece in PNG format.
- **hasMoved**<boolean>: Whether the piece has moved so far. Used for the pawns' initial move.
- **canPromote**<boolean>: Whether the piece can be promoted.

Implement: When creating the piece, the **createPiece()** function expects 2 inputs: type and color. From this information, the piece value and image are automatically implemented from the **PIECE_DATA** constants.

4.4 Authentication Functions & WebSocket Events

Authentication: login, register, forgetPassword, resetPassword, updateUser, deleteUser

Functionality exposed by the **BetterAuth** library related to user authentication, allowing for secure handling of user accounts and session management.

- **login()**: Authenticates the user by validating credentials (email and password) and creating a session token upon success to preserve the authentication state.
- **register()**: Creates a new user account by securely storing user details, which include an email username and hashing passwords.
- **forgetPassword()**: Sends a password reset email to the user's registered email address with a secure token.
- **resetPassword()**: Verifies the reset token and allows the user to set a new password.
- **updateUser()**: Updates user information, such as username, email, or profile settings.
- **deleteUser()**: Permanently removes the user's account and related data from the database.

Game WebSocket Events: join, start, leave, move, end

Real-time communication events for multiplayer gameplay, ensuring synchronization between players.

- **join**: Adds a player to a game session room and broadcasts the updated player list to all connected clients within that room, identified by the game ID.
- **start**: Starts the game when the other player joins, broadcasts the event to both players, with data containing the game id and players list.
- **leave**: Removes a player from the game session, ending the game in a loss for that player if the game was in session; otherwise, simply broadcasts to the other player in the game.
- **move**: Broadcasts a player's move to the other player, updating the game state in real time.
- **end**: Broadcast when the game ends for either client (checkmate, draw, resignation, etc.)

5. User Interface (UI)

Page Layouts:

- **Authentication**: Pages that take up the entire viewport and have forms that are positioned at the center of the page.
- **Main**: The remaining pages feature a sidebar layout, with a fixed sidebar at the left of the page with quick links to other pages, the main content taking up the remaining space.

Color Palettes: The primary color pallet of our UI is a combination of green, grey, black, and white. Within this color pallet, users can switch between light and dark modes based on preference. The light mode enforces colors of green, grey, and white, with minimal black, while the dark mode enforces colors of green, grey, and black, with minimal white. The 'enforcing' colors in each mode are mostly seen on backgrounds, buttons, input boxes, etc, while the 'minimal' colors are mostly seen in a few labels.

Font: Inter variable font

UI Components Library: A collection of well-designed and accessible components from <https://ui.shadcn.com/>

Icons: Consistent, clean, and minimal icon set from <https://lucide.dev/>

Appendix A: UI Pages

Auth Layout: Login Page

Welcome back!

Log in to your PentaChess account

Email

someone@example.com

Password

Forgot your password?

.....

Login

Don't have an account? [Register](#)

Welcome back!

Log in to your PentaChess account

Email

someone@example.com

Password

Forgot your password?

.....

Login

Don't have an account? [Register](#)

Auth Layout: Register Page

Welcome

Create your PentaChess account to play online

Email

someone@example.com

Username

someone

Password

.....

Register

Already have an account? [Login](#)

Welcome

Create your PentaChess account to play online

Email

someone@example.com

Username

someone

Password

.....

Register

Already have an account? [Login](#)

13

Auth Layout: Forgot Password Page

Forgot Password

Enter your email to request a password reset

Email

Send Reset Email

Remembered your password? [Login](#)

Forgot Password

Enter your email to request a password reset

Email

Send Reset Email

Remembered your password? [Login](#)

Auth Layout: Reset Password Page

Reset Password

Enter your new password below

New Password

Reset Password

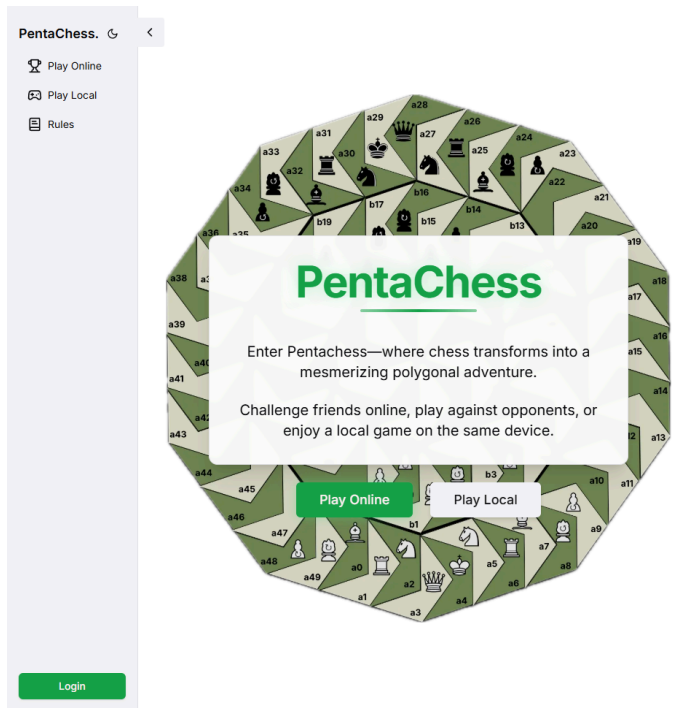
Reset Password

Enter your new password below

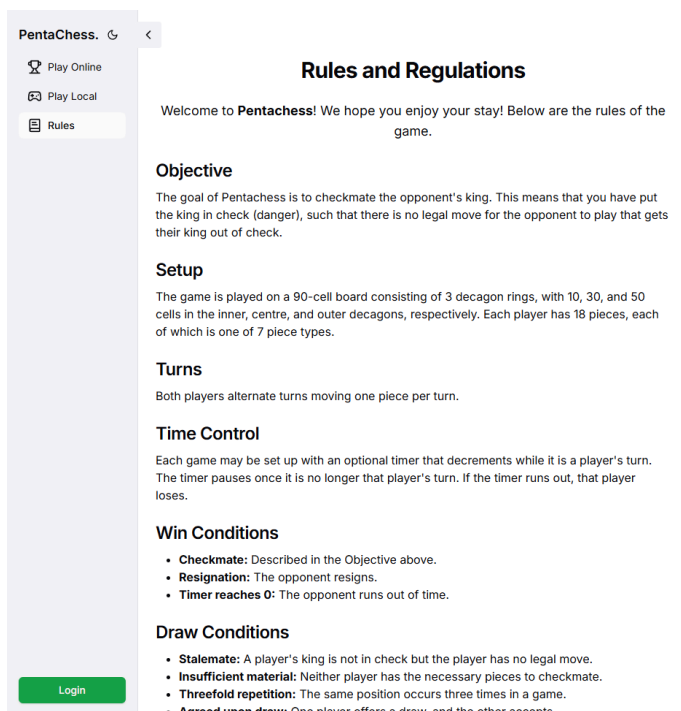
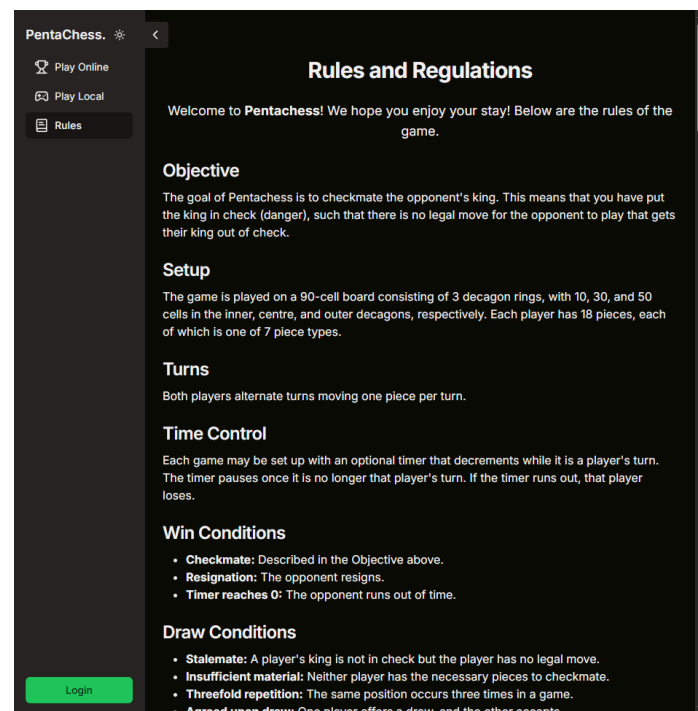
New Password

Reset Password

Main Layout: Home Page



Main Layout: Rules Page



Main Layout: Account Settings Page

PentaChess. ✖

Play Online

Play Local

Rules

Account Settings

Username

This is your username, which will be displayed on your profile and when you play games.

Username

test

Cancel Save

Name

Please enter your full name, or a name you are comfortable with.

Name

John Doe

Cancel Save

Email

Your email address is used to log in and receive password reset emails.

Email

test@mail.com

Cancel Save

T test test@mail...

PentaChess. ⌵

Play Online

Play Local

Rules

Account Settings

Username

This is your username, which will be displayed on your profile and when you play games.

Username

test

Cancel Save

Name

Please enter your full name, or a name you are comfortable with.

Name

John Doe

Cancel Save

Email

Your email address is used to log in and receive password reset emails.

Email

test@mail.com

Cancel Save

T test test@mail...

Main Layout: Create Local Game Page

PentaChess. ✖

Play Online

Play Local

Rules

Create Local Game

Timer Duration (Optional) ☒

Minutes Seconds

Start

Login

PentaChess. ⌵

Play Online

Play Local

Rules

Create Local Game

Timer Duration (Optional) ☒

Minutes Seconds

Start

Login

Main Layout: Create/Join Online Game Page

PentaChess. ✨ <

Play Online

Play Local

Rules

Play Online

Random MatchJoin Game

Find OpponentEnter codeJoin

OR

Create Game

Your Starting Color

randomwhiteblack

Timer Duration (Optional) ☐

Password (optional)

Start

T test

test@mail...

PentaChess. ☁ <

Play Online

Play Local

Rules

Play Online

Random MatchJoin Game

Find OpponentEnter codeJoin

OR

Create Game

Your Starting Color

randomwhiteblack

Timer Duration (Optional) ☐

Password (optional)

Start

T test

test@mail...

Main Layout: Local Game Page

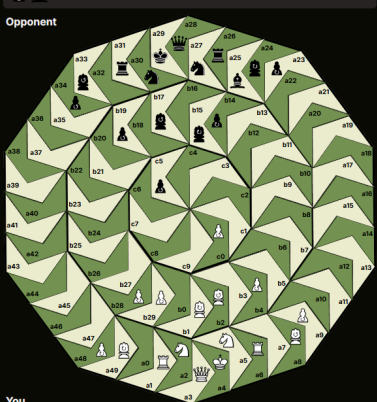
PentaChess. ✨ <

Play Online

Play Local

Rules

Opponent



You



Request DrawResign

1. P-b3-c0P-b18-c5

2. B-a7-b5B-a32-b20

3. B-b5xb15L-b16xb15

4. B-a0-b28B-b20xb28

5. P-b0xb28

Login

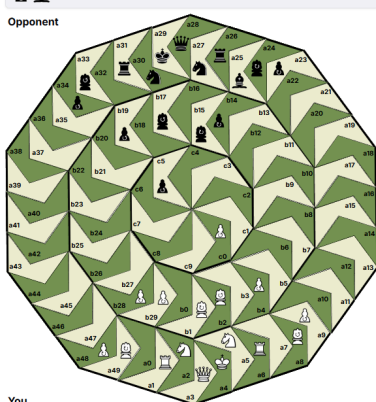
PentaChess. ☁ <

Play Online

Play Local

Rules

Opponent



You



Request DrawResign

1. P-b3-c0P-b18-c5

2. B-a7-b5B-a32-b20

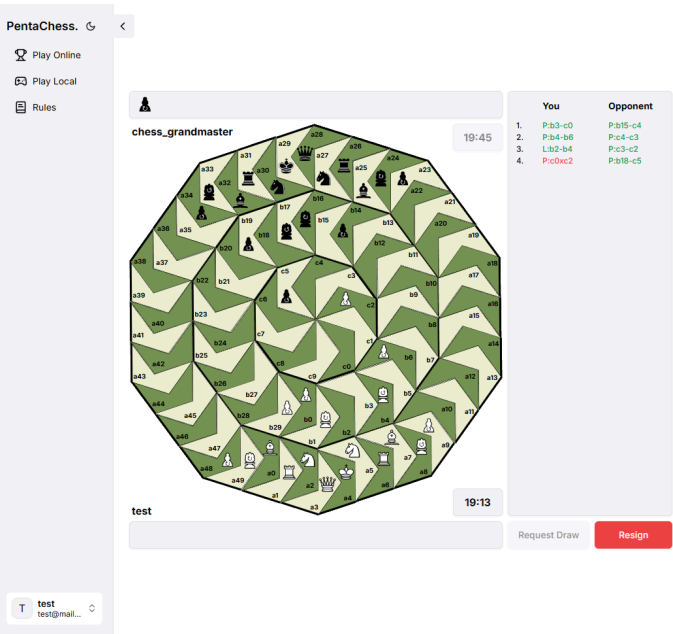
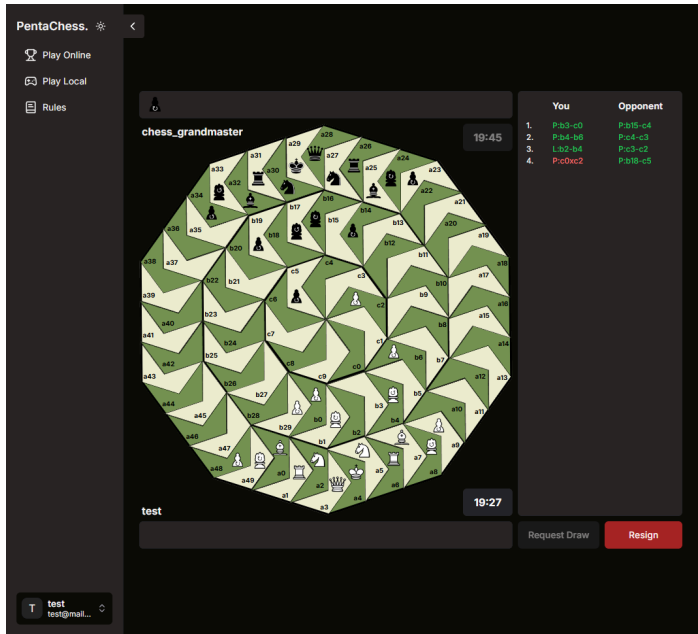
3. B-b5xb15L-b16xb15

4. B-a0-b28B-b20xb28

5. P-b0xb28

Login

Main Layout: Online Game Page (Random Matchmaking)



Not Found Page

